Universidade de São Paulo
Escola Politécnica - Engenharia de Computação e Sistemas Digitais

Linear Regression, Logistic Regression and Nearest Neighbors

Prof. Artur Jordão

Linear Regression

Regression

Linear Regression

- Regression refers to a set of methods for modeling the relationship between one or more independent variables and a dependent variable

- Linear regression
  - The simplest and most popular among the standard tools for regression

Assumptions of Linear Regression

Linear Regression

- Linearity assumption
  - The relationship between the independent variables x and the dependent variable y is linear
  - We can express y can as a **weighted sum** of the elements in x, given some noise on the observations

Linear Model

Linear Regression

- $\hat{y} = w_1 * x^1 + w_2 * x^2 + \cdots + w_m * x^m + \boldsymbol{b}$
  - $W = w_1, w_2, \ldots, w_m$
  - We can organize W as column or row matrix

- Dot product form (single sample prediction)
  - $\hat{y} = xW^T + b$

- Matrix vector product form (n samples prediction – all at once)
  - $\hat{Y} = XW^T + b$

The Bias Term

Linear Regression

- The bias term plays a role in the expressivity of the model
  - It allows the model to fit not only the data points that **pass through the origin but also those that do not**
  - It enables the model to capture and represent more complex relationships between the features and the target variable
  - Animated Gif

Hands-on

The Bias Term

Linear Regression

- Putting the bias into the parameter matrix w_i
  - We can subsume the bias b into the parameter matrix W by appending a column to the design matrix consisting of all **ones**

X			Y (observable)
1.69	10.7	1	5.4
1.85	14.5	1	3.2
2.67	20.3	1	7.1
....	....	...	.
1.62	6.1	1	9.1
1.11	4.0	1	8.7

X			$\hat{Y}$ (unseen)
1.33	8.7	1	?
2.78	3.5	1	?
2.29	7.3	1	?
....	....	...	.
1.62	9.1	1	?
2.15	10.0	1	?

Least Squares

Problem Definition

Least Squares

- How can we discover W (and b) that minimizes the total loss across all training examples?
  - Formally: $\mathrm{W}^*, b^* = argmin\ \mathcal{L}(W, b) \Rightarrow \frac{1}{n}\sum_{i=1}^{n}\frac{1}{2}\left(\overbrace{\boldsymbol{x_i W^T} + \boldsymbol{b}}^{\hat{y}} - y_i\right)^2$
  - Here $\mathcal{L}(\cdot)$ means a loss function

- Techniques
  - Analytical Solution (Ordinary Least Squares – OLS)
  - Gradient-Based Optimization

Analytical Solution

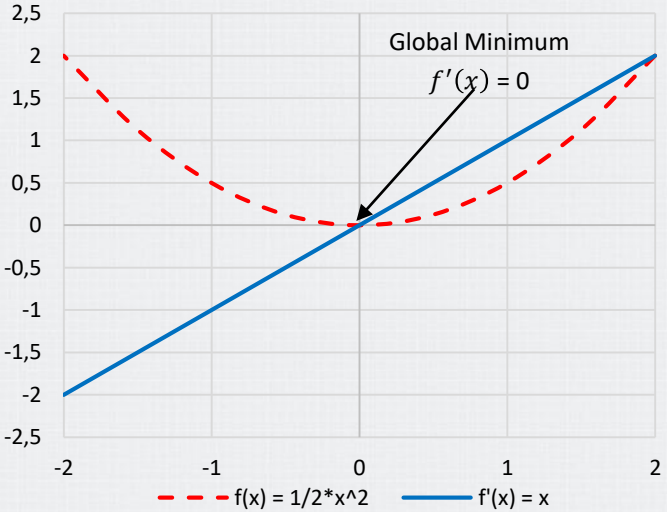
Least Squares

- $\mathcal{L}(W) = \|XW^T - Y\|_2^2$
  - The loss across all training samples

- $\nabla_w \mathcal{L}(W) = 2X^T(XW^T - Y) = 0$
  - We set the gradient to zero to find the points where the loss function reaches the minimum

- $W^* = \underbrace{(X^T X)^{-1} X^T}\, Y$

  The **pseudoinverse** of X

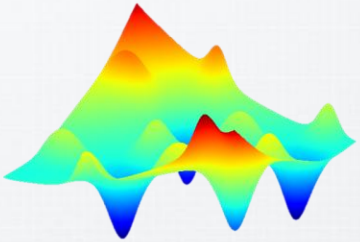


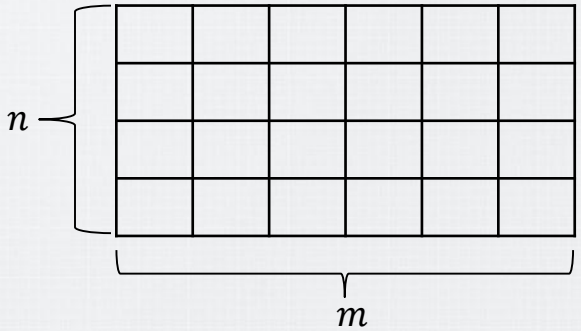
Analytical Solution

Least Squares

- Problems
  - It does not work for complex (non-convex – multiple local optima) problems

- The sample size problem (a.k.a zero determinant or singularity)
  - The number of samples is smaller than the number of features ($n < m$)



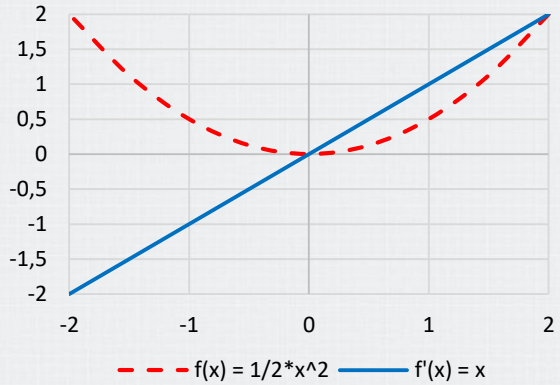


Hands-on

Gradient Descent

Least Squares

- The gradient is useful for minimizing a function
  - It tells us how to change x in order to make a small improvement in y
- We can reduce $f(x)$ by moving x in small steps with **opposite sign of the gradient**
- The key consists of **iteratively** reducing the error by updating the parameters (weights) in the direction that incrementally lowers the loss function
  - **Gradient Descent**



Gradient Descent

Least Squares

- Learning rate (η)
  - Positive scalar determining the size of the step
  - The rate of learning

- Convergence
  - Run for a defined number of iterations (**epochs**)
  - Stop when the loss does not decrease (early stop)

Gradient Descent Algorithm

$\boldsymbol{W} \leftarrow$ Random values

While not converged do

for each $\boldsymbol{w_i} \in \boldsymbol{W}$ do

$$\boldsymbol{w_i} \leftarrow \boldsymbol{w_i} - \eta \frac{\partial}{\partial \boldsymbol{w_i}} \mathcal{L}(\boldsymbol{W})$$

Hands-on

Gradient Descent

Least Squares

- The Gradient Descent takes the derivative of the loss function
  - It computes the **average of the losses** over every single example ($x \in X$)

- In practice, this can be extremely slow and memory costly
  - We must pass over the entire dataset before making a single update
  - The problem is compounded if n is larger than the processor's memory size

Stochastic Gradient Descent (SGD)

Least Squares

- Stochastic Gradient Descent randomly selects a small number (**batch size** – β) of training examples at each step t, and updates according to

$$W \leftarrow W - \eta \frac{1}{|\beta|} \sum_{j=\beta_t}^{n} \frac{\partial}{\partial \boldsymbol{W}} \mathcal{L}^j(\boldsymbol{W})$$

Stochastic Gradient Descent Algorithm

$\boldsymbol{W} \leftarrow$ Random values

While not converged do

for each $\boldsymbol{w_i} \in \boldsymbol{W}$ do

$$\boldsymbol{w_i} \leftarrow \boldsymbol{w_i} - \eta \frac{1}{|\beta|} \sum_{j=\beta_t}^{n} \frac{\partial}{\partial \boldsymbol{w_i}} \mathcal{L}^j(\boldsymbol{W})$$

Logistic Regression

Introduction

Logistic Regression

- How can we learn a linear classification model instead of a regression model?

- A naïve solution is passing the linear function xW^T through a threshold function (th)
  - $\begin{cases} \hat{y} = 1 \; if \; xW^T \geq th \\ \hat{y} = 0, otherwise \end{cases}$
  - **Hard threshold**

- Despite simple, the hard nature of the threshold causes some problems
  - The optimization function is not differentiable
  - The linear classifier is always fully confident (0 or 1), even near the decision boundary; ideally, it should mark some cases as uncertain

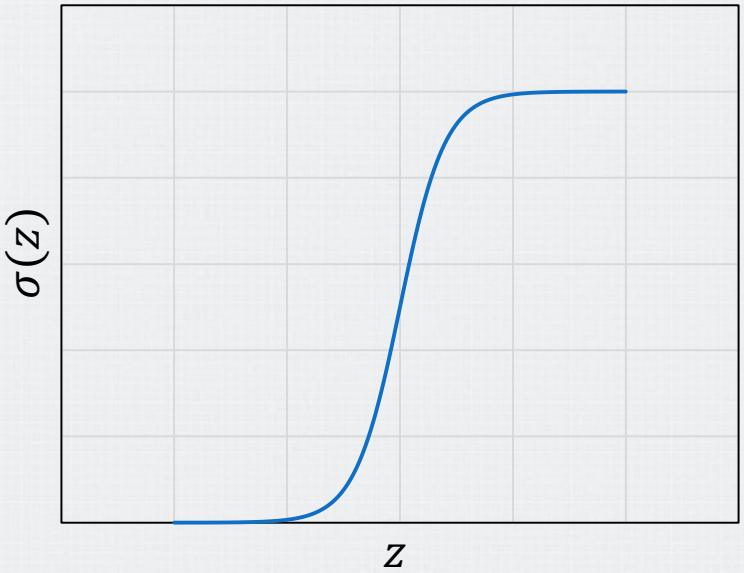
Problem Definition

Logistic Regression

- To learn a linear classification model, we soften the threshold function—approximating the hard threshold with a continuous, differentiable function

- Logistic function (a.k.a sigmoid – $\sigma(.)$)
  - $Logistic(xW^T) = \frac{1}{1+e^{-xW^T}}$
  - $\sigma(z) = \frac{1}{1+e^{-z}}$

- This modeling approach defines a soft boundary, assigning probability 0.5 at the center and approaching 0 or 1 away from it



Logistic Regression Classifier

Logistic Regression

- Because of the **nonlinearity** of the sigmoid function, we cannot solve directly
  - There is no easy closed-form solution to find the optimal value of W

- Fortunately, we can apply the gradient descent optimization
  - The derivative of $\sigma(z) = \frac{1}{1+e^{-z}}$ is $\sigma'(z) = \sigma(z)(1-\sigma(z))$

Gradient Descent Algorithm

$\boldsymbol{W} \leftarrow$ Random values

While not converged do

for each $\boldsymbol{w_i} \in \boldsymbol{W}$ do

$$\boldsymbol{w_i} \leftarrow \boldsymbol{w_i} - \eta \frac{\partial}{\partial \boldsymbol{w_i}} \mathcal{L}(\boldsymbol{W})$$

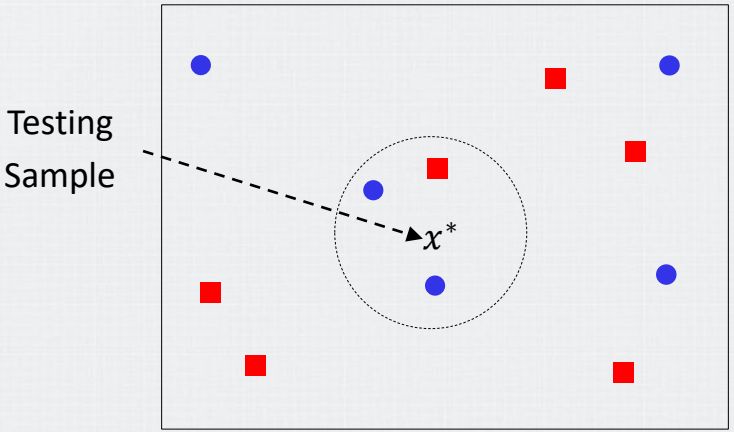
$\mathcal{L}$	**Update** ($\hat{y} = \sigma(xW^T)$)
L_2	$w_i \leftarrow w_i - \eta(\boldsymbol{y} - \hat{\boldsymbol{y}})\hat{\boldsymbol{y}}(\boldsymbol{1} - \hat{\boldsymbol{y}})\boldsymbol{x}$
Cross-entropy	$w_i \leftarrow w_i - \eta(\hat{\boldsymbol{y}} - \boldsymbol{y})\boldsymbol{x}$

Nearest Neighbors

Introduction

Nearest Neighbors

- A very simple classifier that works surprisingly well on many problems
- The nearest neighbor classifier assigns an instance (testing sample x^*) to the class most heavily represented among its neighbors
- It leverages the idea that the more similar the instances, the more likely it is that they belong to the same class



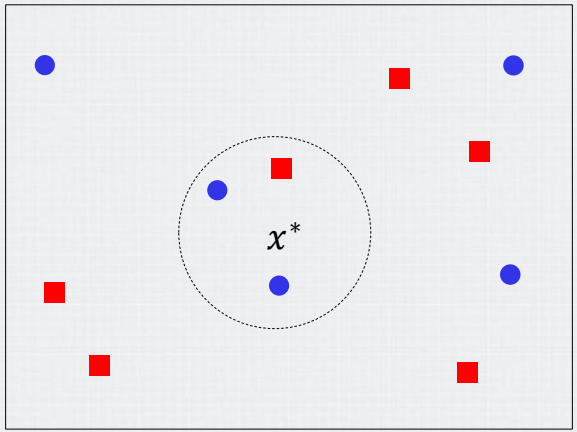
Overview

Nearest Neighbors

- Requirements
  - A positive a positive integer k, a set of labeled samples (training data)
  - A metric to measure *closeness* (e.g., Euclidean or cosine)

- For a given unlabeled example x^*, find the k closest labeled examples in the training data set (nearest neighbor search) and assign $\hat{y}$ to the most frequent class among the k neighbors

Testing Sample



Overview

Nearest Neighbors

- Assuming X^{NN} as the set of k nearest neighbors of x_i
  - $\Pr(Y = class\ j \mid X = x_i) = \frac{1}{k}\sum_{i \in X^{NN}} I(y_i = class\ j)$
  - I stands for the indicator function $\begin{cases} 1\ if\ y_i = class\ j \\ 0, otherwise \end{cases}$
  - Observe that it is predicting the mode of the labels among the k nearest neighbors

- For regression
  - $\hat{y} = \frac{1}{k}\sum_{i \in X^{NN}} y_i$

Practical Issues

Nearest Neighbors

- Contrary to other learning algorithms that allow discarding the training data after the model is built, kNN keeps **all training examples in memory**
  - We can speed up the NN search using k-d trees

- The parameter k plays a role on the kNN classifier
  - With $k = 1$, the training error rate is 0. Low bias but very high variance
  - As k grows we have low-variance but high-bias
  - To avoid ties on binary classification, a common practice is assigning k an odd number

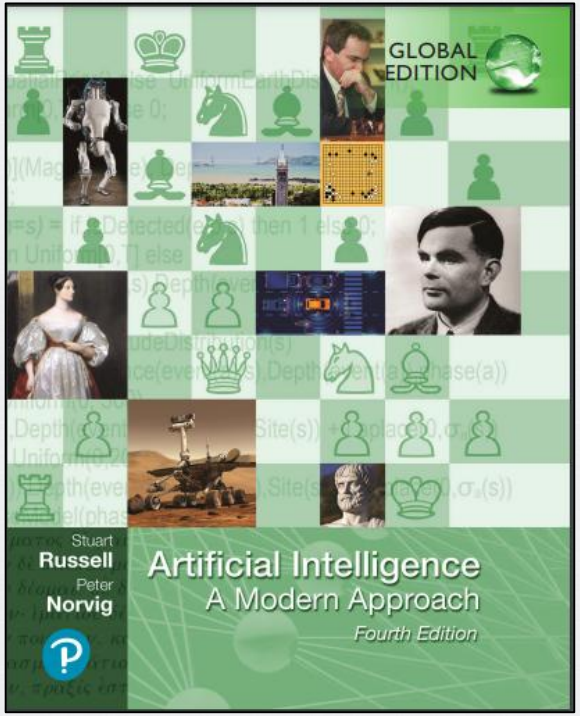
- Highly susceptible to the curse of dimensionality

Hands-on

Bibliography

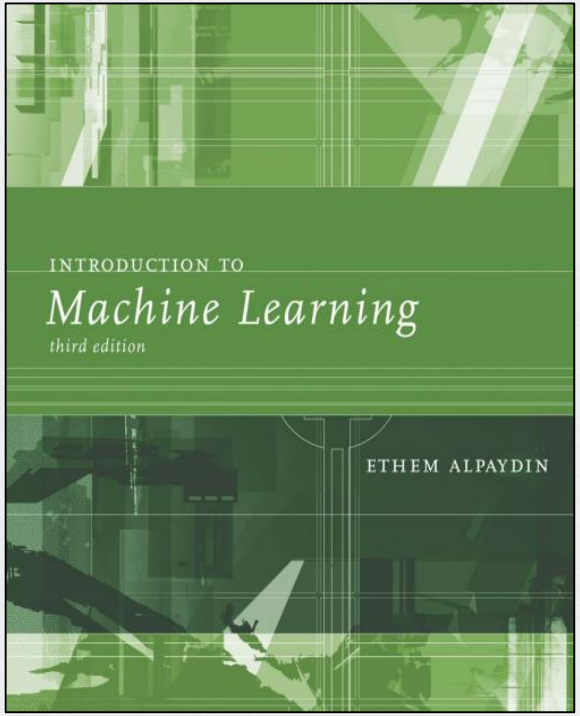
Bibliography

- Artificial Intelligence A Modern Approach
  - Chapter 19
    - 19.6.5 Linear classification with logistic regression
    - 19.7.1 Nearest-neighbor models



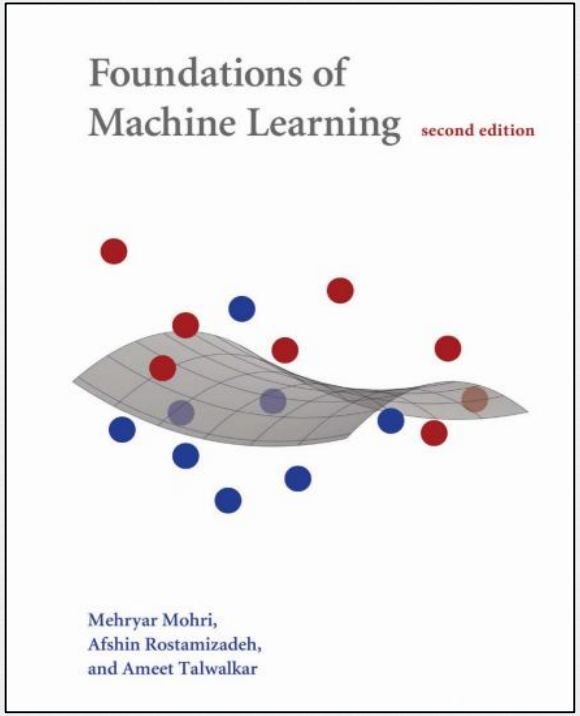
Bibliography

- INTRODUCTION TO Machine Learning
  - Chapter 8
    - 8.4 Nonparametric Classification
  - Chapter 10
    - 10.5 Parametric Discrimination Revisited
    - 10.7 Logistic Discrimination



Bibliography

- Foundations of Machine Learning
  - Chapter 13
    - 13.7 Logistic regression



Bibliography

- The Hundred-Page Machine Learning
  - Chapter 3
    - 3.2 Logistic Regression
    - 3.5 k-Nearest Neighbors